LevelUp Cloud Trainings
Sateesh Pabbathi | Microsoft Certified Trainer (MCT)

PySpark

A Complete Beginner's Guide

Why PySpark? | Architecture | Terminology | Data Distribution | RDDs & DataFrames | Execution Model

Why PySpark	Architecture	Data Model	Distribution	Execution
Foundation	Driver & Executors	RDD & DataFrame	Partitions & Shuffle	DAG & Stages

www.levelupedu.net

ABOUT THE AUTHOR

Microsoft Certified Trainer | Data Engineering Educator | 22+ Years Industry Experience

Sateesh Pabbathi
MCT
www.levelupedu.net

Sateesh Pabbathi

Microsoft Certified Trainer (MCT)

With 22+ years of industry experience in data and cloud technologies, Sateesh Pabbathi is a renowned Microsoft Certified Trainer (MCT) and the founder of LevelUp Cloud Trainings — India's premier data engineering education platform.

Having trained 5,000+ students, Sateesh is famous for his story-driven, analogy-based teaching that makes PySpark — one of the most feared technologies — crystal clear even for zero-experience beginners.

His Indian business analogies — Zomato, Flipkart, HDFC Bank, Infosys, TCS — make distributed computing feel as familiar as ordering food online!

Core Expertise

- Microsoft Fabric (DP-600) | Apache Spark & PySpark
- Databricks | Azure Data Factory | GCP & AWS
- SQL Server | Python | Big Data & Hadoop Ecosystem

"I don't just teach technology — I build careers."

5,000+	22+	10+
Students Trained	Years Experience	Technologies Taught

Module 1: Why PySpark? The Big Picture

Before touching a single line of code, let's answer the most important question: WHY does PySpark even exist? Understanding the WHY makes the HOW 10x easier to learn.

> **Sateesh Says** 💡
> *Imagine you are the CTO of Flipkart. You have 10 TB of customer purchase data processed every night.*
> *You try Python + Pandas. It crashes. RAM is only 16 GB. Your laptop gives up!*
> *You try SQL on a single server. It takes 18 hours. Business wants results in 30 minutes!*
> *This is exactly why PySpark was born — to process data that doesn't fit on ONE machine.*

1.1 The Problem PySpark Solves

Python (with Pandas) is fantastic for small data — up to a few GB on a single machine. But real-world enterprise data is huge. PySpark gives Python developers a way to process terabytes and petabytes across a cluster of machines — using familiar Python syntax.

The Scale Problem — Why a Single Machine Fails

```
Your Pandas Code on a Laptop:
+-----------------------------------------+
| df = pd.read_csv('orders_10TB.csv')   <- | CRASH! MemoryError
| df.groupby('city').sum()              <- | Takes hours if run
+-----------------------------------------+
RAM: 16 GB    |    Data: 10,000 GB (10 TB)   =   IMPOSSIBLE

PySpark on a 100-node Cluster:
+--------+ +--------+ +--------+       +--------+
| Node 1 | | Node 2 | | Node 3 |  ...  | Node100|
| 64 GB  | | 64 GB  | | 64 GB  |       | 64 GB  |
| 100 GB | | 100 GB | | 100 GB |       | 100 GB |
| data   | | data   | | data   |       | data   |
+--------+ +--------+ +--------+       +--------+
     |          |          |                 |
     +----------+----------+----...----------+
                           |
         ALL NODES WORK IN PARALLEL -> Result in minutes!
Total RAM: 100 x 64 GB = 6,400 GB  >>  Handles 10 TB easily!
```

1.2 What Exactly is PySpark?

- **Apache Spark:** Open-source distributed computing engine written in Scala. Processes massive data at speed.
- **PySpark:** The Python API for Apache Spark. You write Python — Spark runs on a cluster underneath.
- **Key insight:** PySpark code looks like Python, but it runs on hundreds of machines simultaneously.

Python + Pandas	PySpark
Single machine only	Distributed cluster (100s of nodes)
Processes data in RAM — GBs	Processes data across cluster RAM — PBs
Sequential operations	Parallel processing on all nodes at once
Best for: data up to ~10 GB	Best for: 10 GB to Petabytes
No fault tolerance	Automatic fault tolerance via lineage
Runs anywhere, free	Requires cluster: Databricks, YARN, K8s

1.3 PySpark vs Hadoop MapReduce

Sateesh Says 💡

MapReduce is like sending letters by India Post — it works, but writes to disk after every step. SLOW!

PySpark is like WhatsApp — everything happens in memory (RAM), so it is 100x faster!

Spark did NOT replace Hadoop. It replaced MapReduce. HDFS is still used as storage under Spark!

MapReduce vs PySpark — Processing Comparison

```
TASK: Compute average order value per city (100 GB dataset)

MAPREDUCE: Disk-Based (each step reads and writes disk)
Read -> [MAP] -> Write Disk -> [SHUFFLE] -> Write Disk
     -> [REDUCE] -> Write Disk -> Output
Time: ~45 minutes. Each step hits HDFS. SLOW!

PYSPARK: In-Memory (data stays in RAM between steps)
Read -> [Transform] -> RAM -> [Transform] -> RAM -> Write
Only disk I/O at start and end!
Time: ~3 minutes. 100x faster for iterative algorithms!

Why RAM beats Disk:
Disk Speed: 100-500 MB/s    |   RAM Speed: 10,000-50,000 MB/s
```

Module 2: PySpark Architecture — The Full Picture

Understanding PySpark architecture is the foundation of everything. Know this well and every other concept falls into place.

> **Sateesh Says 💡**
> *Think of PySpark like a large construction project — the Infosys campus in Pune.*
> *The Project Manager (Driver) plans all work, creates tasks, monitors progress.*
> *The Construction Workers (Executors) actually do the work — lay bricks, pour concrete.*
> *The Project Manager doesn't carry a single brick! Workers don't plan — they just execute.*
> *Driver plans. Executors execute. Beautiful separation of responsibility!*

2.1 PySpark Architecture — Master Diagram

PySpark Complete Architecture

```
YOUR CODE (Python Script / Jupyter / Databricks Notebook)
      |
      v
+-----------------------------------------------------------------+
|                      DRIVER PROGRAM                             |
|                   (The Brain / Master)                          |
|  +-------------------------------+  +---------------------------+ |
|  |   SparkSession                |  |   DAG Scheduler           | |
|  |   (Entry point to Spark)      |  |   (Plans execution)       | |
|  |   Job -> Stages -> Tasks      |  |   Optimizes your queries  | |
|  +-------------------------------+  +---------------------------+ |
+-----------------------------------------------------------------+
      |                 |                  |
      v                 v                  v
+-----------+    +-----------+       +-----------+
|   YARN    |    |   Mesos   |       | Kubernetes|  <-- Pick ONE
|  Cluster  |    |  Cluster  |       | (K8s)     |    Cluster Mgr
|  Manager  |    |  Manager  |       |  Manager  |
+-----------+    +-----------+       +-----------+
      |   Allocates CPU + RAM to Executors on Worker Nodes
      v
+-----------------------------------------------------------------+
|                      WORKER NODES                               |
|  +------------------+  +------------------+  +--------------+   |
|  |   Executor 1     |  |   Executor 2     |  |  Executor N  |   |
|  |  (Worker Process)|  |  (Worker Process)|  | (Worker Proc)|   |
|  |   Task1 | Task2  |  |   Task3 | Task4  |  |   TaskN      |   |
|  |   Slot  | Slot   |  |   Slot  | Slot   |  |   Slot       |   |
|  +------------------+  +------------------+  +--------------+   |
|  Machine A            Machine B            Machine C             |
+-----------------------------------------------------------------+
                           |
                           v
                 HDFS / Azure ADLS / AWS S3 (Storage)
```

2.2 Key Architecture Components

Driver Program

The master process of any Spark application.
Runs your main Python function / notebook.
Creates SparkSession (entry point to Spark).
Converts your code into a DAG execution plan.
Sends Tasks to Executors via Cluster Manager.
Analogy: Project Manager at TCS Pune

Executor

Worker process that runs on each cluster node.
Receives Tasks from the Driver.
Runs actual computation on data partitions.
Stores results in memory (cache) or disk.
Reports task status back to Driver.
Analogy: Construction worker on site

SparkSession

Unified entry point in Spark 2.x and above.
Replaces old SparkContext, SQLContext, HiveContext.
Start with: spark = SparkSession.builder...
Manages Spark configuration and cluster conn.
Your gateway to all Spark functionality.
Analogy: Your employee ID card at Infosys

Cluster Manager

Resource allocator for the cluster.
Decides which nodes run your Executors.
Types: YARN, Mesos, Kubernetes, Standalone.
On Databricks — all managed automatically.
Driver asks Cluster Manager for resources.
Analogy: HR dept assigning staff to projects

2.3 Execution Flow — What Happens When You Run a Query

Step-by-Step: df.groupBy('city').count().show()

```
STEP 1: You submit your PySpark script / run the cell
-> Driver process starts on master node

STEP 2: SparkSession is created
-> spark = SparkSession.builder.getOrCreate()
-> Driver connects to Cluster Manager

STEP 3: Driver reads transformations (LAZY -- nothing runs yet!)
-> df = spark.read.csv('orders.csv')   -- builds a plan only
-> df2 = df.groupBy('city').count()    -- adds to the plan
-> df2.show()                          -- ACTION! Execute now!

STEP 4: Driver builds the DAG (execution plan)
Read CSV --> GroupBy city --> Count --> Show result
Catalyst Optimizer rewrites plan for maximum efficiency

STEP 5: DAG Scheduler splits DAG into Stages
Stage 1: Read CSV, scan partitions (no shuffle needed)
Stage 2: GroupBy + Count (shuffle -- data moves between nodes!)

STEP 6: Tasks sent to Executors via Cluster Manager
Each Executor processes one partition independently in parallel

STEP 7: Executors return results -> Driver aggregates -> show()
```

Module 3: Core Terminology — Know Your Vocabulary

In PySpark, terminology is everything. Knowing the right words — and using them correctly — is what separates junior developers from senior data engineers in interviews at Amazon, Flipkart, and TCS.

> **Sateesh Says 💡**
> *When you say 'I used groupBy' — that is junior level.*
> *When you say 'groupBy triggered a shuffle, creating a Stage boundary. I tuned spark.sql.shuffle.partitions from 200 to 400 to match our cluster cores' — that is SENIOR level!*
> *Learn the words. Use them correctly. Your salary will thank you!*

3.1 The Core PySpark Vocabulary

Term	Definition & Real-World Context
Job	One complete PySpark application execution. Every time you call an action (.show(), .count(), .write()), Spark creates a Job. Like one complete project delivery at Infosys.
Stage	A Job is divided into Stages at shuffle boundaries. Stage 1 might read + filter. Stage 2 might groupBy + aggregate. Stages within a job can run in parallel where possible.
Task	Smallest unit of work. Each Stage is split into Tasks — one Task per Partition. If a Stage has 200 partitions, it runs 200 Tasks. Tasks run in parallel across Executor slots.
Partition	A logical chunk of your data. A 10 GB file might become 80 partitions. Each partition is processed by one Task on one Executor core. More partitions = more parallelism.
RDD	Resilient Distributed Dataset — Spark's original data structure. Immutable distributed collection. Low-level API. Rarely used directly today but foundational for understanding Spark internals.
DataFrame	Distributed table with named, typed columns — like a Pandas DataFrame but across a cluster. Built on RDD. 99% of modern PySpark code uses DataFrames. Optimized by Catalyst.
Transformation	Operation that creates a new DataFrame. LAZY — not executed immediately! Builds the execution plan. Examples: filter(), select(), groupBy(), join(), withColumn(), orderBy().
Action	Operation that TRIGGERS execution of all pending transformations. Returns a result. Examples: show(), count(), collect(), take(), first(), write(), saveAsTable(). Actions = Spark works!
Lazy Evaluation	Spark does NOT execute transformations immediately. Waits for an Action, then

	optimizes the full chain and executes. Like a chef who plans the entire multi-course meal before cooking any dish!
DAG	Directed Acyclic Graph — Spark's execution plan. Visual map of all transformations in order. No cycles. Optimized by Catalyst Optimizer. Divided into Stages at shuffle boundaries.
Shuffle	Movement of data between partitions across the network. Triggered by groupBy, join, distinct, repartition. Very expensive (network + disk I/O). Minimize shuffles for fast PySpark code!
Lineage	History of how an RDD/DataFrame was created — every transformation applied. Used for fault tolerance: if a partition is lost, Spark re-computes it from lineage. No replication needed!
Cache / Persist	Stores a DataFrame in memory (or disk) to avoid recomputation. Use when you access the same DataFrame multiple times. cache() = memory+disk. persist() lets you choose storage level.
Broadcast Variable	A read-only variable cached on EVERY Executor. Avoids shuffling large lookup tables. Use: df.join(broadcast(small_df), 'city'). One of the biggest performance tricks in PySpark!
Catalyst Optimizer	Spark's built-in query optimizer. Analyzes your logical plan, applies rules (predicate pushdown, column pruning, join reordering), creates optimized physical plan. Why DataFrame > RDD in speed.
Tungsten Engine	Spark's physical execution engine. Manages memory at binary level, bypasses JVM overhead, generates optimized JVM bytecode. Responsible for Spark's raw processing speed.

3.2 Transformation vs Action — The Most Important Distinction

Lazy Evaluation — The Chef Analogy

```
You write 4 transformations:
df1 = spark.read.csv('orders.csv')     # Transformation - lazy
df2 = df1.filter(df1.amount > 100)     # Transformation - lazy
df3 = df2.groupBy('city')              # Transformation - lazy
df4 = df3.agg({'amount': 'sum'})       # Transformation - lazy
df4.show()                              # ACTION! Spark starts!

Nothing happens until .show() is called!

WHY lazy evaluation is powerful:
-> Spark sees the FULL picture before executing
-> Catalyst removes unused columns early (column pruning)
-> Filters are pushed down to read less data from disk
-> Multiple steps combined into fewer actual operations

Like ordering Biryani + Raita + Dessert at once. The chef plans
the full order, uses the same boiling pot for rice and gravy,
and doesn't cook each dish independently. Smart and efficient!
```

Module 4: How Data is Distributed — Partitions Deep Dive

This is the most misunderstood concept in PySpark. Understanding partitions separates developers who write 'working code' from engineers who write 'fast code'. Master this and performance tuning becomes intuitive.

> **Sateesh Says 💡**
>
> *Imagine Swiggy has 10,000 food orders in Hyderabad. They don't send ONE delivery boy!*
>
> *They split the city into zones: Banjara Hills (500 orders), Madhapur (800 orders), Gachibowli (600 orders)...*
>
> *Each zone has dedicated delivery boys working SIMULTANEOUSLY. 10,000 orders delivered in 30 minutes!*
>
> *PySpark partitions your data into chunks. Each chunk is processed by one Task on one Executor. Parallel magic!*

4.1 What is a Partition?

A partition is a logical horizontal slice of your dataset. When PySpark reads a file from HDFS or S3, it automatically splits it into partitions. Each partition is processed independently by one Task on one Executor core — all in parallel.

Partitioning — Visual Explanation

```
  File: orders.csv = 800 MB on HDFS (6 HDFS blocks x ~128 MB each)
  PySpark automatically creates 6 Partitions from 6 HDFS blocks:

  +-------------------------------------------------------------+
  |         orders.csv  (800 MB, 10 million rows)               |
  +-------------------------------------------------------------+
       |         |         |         |         |        |
       v         v         v         v         v        v
  +-------+ +-------+ +-------+ +-------+ +------+ +------+
  |Part-0 | |Part-1 | |Part-2 | |Part-3 | |Part-4| |Part-5|
  |128 MB | |128 MB | |128 MB | |128 MB | |128 MB| |128 MB|
  |1.67M  | |1.67M  | |1.67M  | |1.67M  | |1.67M | |1.67M |
  |rows   | |rows   | |rows   | |rows   | |rows  | |rows  |
  +-------+ +-------+ +-------+ +-------+ +------+ +------+
      |         |         |         |         |        |
      v         v         v         v         v        v
 [Task 0]  [Task 1]  [Task 2]  [Task 3]  [Task 4] [Task 5]
 Exec1     Exec1     Exec2     Exec2     Exec3    Exec3
 Core 1    Core 2    Core 1    Core 2    Core 1   Core 2

  All 6 Tasks run IN PARALLEL!  Total time ~= 1/6th of sequential!
```

4.2 Partition Count — The Goldilocks Problem

Too few partitions = underutilized cluster (idle cores). Too many partitions = scheduling overhead. Finding the right number is critical.

Too Few vs Too Many vs Just Right

```
SCENARIO: 100-core cluster, processing 10 GB of data

TOO FEW (2 partitions):
[Partition 0: 5 GB] [Partition 1: 5 GB]
Task 0 uses 1 core. Task 1 uses 1 core.
98 cores sitting IDLE. Massive resource waste!

TOO MANY (10,000 partitions):
10,000 x 1 MB partitions. 10,000 Tasks to schedule!
Task scheduling overhead > actual computation time.
Driver overwhelmed managing 10,000 tiny tasks.

JUST RIGHT (200-400 partitions):
Rule of thumb: 2 to 4 partitions per CPU core in cluster.
100 cores x 2-4 = 200-400 partitions. Sweet spot!
Each partition: 25-50 MB. Efficient scheduling and processing.

Key config: spark.sql.shuffle.partitions = 200 (default, tune!)
```

4.3 Partitioning Types

Hash Partitioning (Default)	Range Partitioning	Custom Partitioning
Uses hash(key) % numPartitions to assign rows to partitions. Same key always goes to same partition. Used automatically by groupBy and join. May cause skew if key distribution is uneven.	Sorts data and assigns contiguous ranges to partitions. Ensures similar row counts per partition. Used with orderBy and sortBy operations. Better distribution than hash for skewed data.	You define your own partitioning logic. Use when you know your data patterns. Example: partition by date or by region. df.repartition(10, col('city'))

4.4 repartition() vs coalesce() — Critical Interview Difference

Sateesh Says 💡

This question comes up in EVERY PySpark interview at Amazon, Walmart Global Tech, and banking firms!

repartition() = Full shuffle. Can increase OR decrease partitions. Slow but even distribution.

coalesce() = No shuffle. Can only DECREASE partitions. Fast but may create uneven partition sizes.

Rule: Use coalesce() before writing output to reduce small files. Use repartition() to fix data skew.

repartition() vs coalesce() Visual Difference

```
STARTING: 8 Partitions, 100 rows each (800 rows total)

df.repartition(4)  -- Full shuffle, data redistributed evenly
Before: [100][100][100][100][100][100][100][100]
After:  [200]     [200]     [200]     [200]
SHUFFLE: YES. All data crosses the network. Slow but perfectly
balanced. Use when you need even partitions before heavy groupBy.

df.coalesce(4)  -- No shuffle, just merges adjacent partitions
Before: [100][100][100][100][100][100][100][100]
After:  [100+100][100+100][100+100][100+100]
SHUFFLE: NO. Fast! But result may be uneven for real data.
Use before writing output to reduce number of output files.

KEY: coalesce() cannot INCREASE partitions! Use repartition()
to go from 4 partitions to 16 partitions.
```

Module 5: RDD, DataFrame & Dataset — The Data Models

PySpark has three main data structures. Understanding each, when they were introduced, and when to use them is a key differentiator in senior-level interviews.

> **Sateesh Says 💡**
> *Think of RDD as riding a bicycle — full control, your legs do all the work. Powerful but exhausting!*
> *DataFrame is like driving a car — you just steer. The engine (Catalyst) optimises automatically.*
> *Dataset is like a self-driving car — type safety + auto-optimisation. Java/Scala only though.*
> *In 2025, 99% of PySpark code uses DataFrames. But RDD knowledge is still tested in EVERY interview!*

5.1 RDD — Resilient Distributed Dataset

The original Spark data structure introduced in 2012. Low-level, powerful, verbose. RDD is the raw engine underneath everything else in Spark.

- **Resilient:** If a partition is lost (node crash), Spark rebuilds it from lineage. Automatic recovery.
- **Distributed:** Split across multiple nodes. Each partition lives on a different machine.
- **Dataset:** A collection of Python objects (not structured rows like DataFrame).

RDD Lineage — How Fault Tolerance Works

```
RDD_A = sc.textFile('orders.csv')         [Original data on HDFS]
    |
    v  .filter(lambda x: 'Mumbai' in x)
RDD_B = RDD_A.filter(...)                  [Filtered RDD]
    |
    v  .map(lambda x: x.split(',')[2])
RDD_C = RDD_B.map(...)                     [Mapped RDD]

SCENARIO: Node crash! Partition 3 of RDD_C is LOST.

Spark checks lineage graph:
RDD_C Part3 was built from RDD_B.map()
RDD_B Part3 was built from RDD_A.filter()
RDD_A Part3 is still safe on HDFS (replicated!)

Spark re-runs: filter() -> map() on RDD_A Part3
RDD_C Partition 3 is RESTORED. Zero data loss!
No replication of RDD needed -- lineage handles recovery!
```

5.2 DataFrame — The Modern Way (Use This Always)

Introduced in Spark 1.3. A DataFrame is a distributed table with named, typed columns. Spark's Catalyst Optimizer automatically optimizes DataFrame operations — often making badly written DataFrame code faster than well-written RDD code.

DataFrame Structure and Sample Operations

```
df = spark.read.csv('employees.csv', header=True, inferSchema=True)

df.printSchema():
root
 |-- emp_id:   integer  (nullable = true)
 |-- name:     string   (nullable = true)
 |-- city:     string   (nullable = true)
 |-- salary:   double   (nullable = true)
 |-- dept:     string   (nullable = true)

df.show(3):
+--------+-----------+-----------+--------+-----------+
| emp_id | name      | city      | salary | dept      |
+--------+-----------+-----------+--------+-----------+
| 1001   | Raj Kumar | Mumbai    | 85000  | Analytics |
| 1002   | Priya S   | Delhi     | 92000  | DataEng   |
| 1003   | Amit V    | Hyderabad | 78000  | Analytics |
+--------+-----------+-----------+--------+-----------+

Key operations:
df.filter(df.city == 'Mumbai')             # Filter rows
df.select('name', 'salary')                # Select columns
df.withColumn('tax', df.salary * 0.30)     # Add new column
df.groupBy('dept').agg({'salary': 'avg'})  # Aggregate
df.orderBy('salary', ascending=False)       # Sort
```

5.3 Spark SQL — Write SQL on Distributed Data

PySpark lets you write pure SQL on DataFrames! Register your DataFrame as a temporary view, then query with SQL. Identical performance to DataFrame API — both go through the Catalyst Optimizer.

PySpark SQL — Two Ways, Same Performance

```
METHOD 1: DataFrame API
result = df.filter(df.city == 'Mumbai') \
            .groupBy('dept') \
            .agg({'salary': 'avg'}) \
            .orderBy('avg(salary)', ascending=False)

METHOD 2: Spark SQL (identical performance!)
df.createOrReplaceTempView('employees')
result = spark.sql('''
    SELECT dept, AVG(salary) as avg_salary
    FROM employees
    WHERE city = 'Mumbai'
    GROUP BY dept
    ORDER BY avg_salary DESC
''')
```

```
Both produce IDENTICAL execution plans via Catalyst Optimizer!
Choose whichever your team prefers -- no performance difference.
```

Module 6: DAG, Stages & Shuffle — Execution Internals

This module takes you from 'PySpark user' to 'PySpark engineer'. Understanding DAGs, stages, and shuffle lets you read Spark UI, diagnose bottlenecks, and write production-grade optimized code.

> **Sateesh Says 💡**
>
> *When your PySpark job runs for 3 hours instead of 3 minutes, how do you find the problem?*
> *You open Spark UI (port 4040) and look at the DAG visualization, Stage timeline, and shuffle bytes.*
> *Engineers who can READ Spark UI get promoted. The others just say 'the job was slow, don't know why'.*
> *Learn DAGs and Stages — they are your X-ray machine for diagnosing slow PySpark code.*

6.1 DAG — Directed Acyclic Graph

A DAG is Spark's execution plan — a graph showing every transformation your data goes through. Directed means there is a flow direction. Acyclic means no loops. The Driver builds the DAG when an Action is triggered.

DAG Example — Flipkart Order Analysis Pipeline

```
Your code:
df  = spark.read.parquet('s3://data/orders/')     [1]
df2 = df.filter(df.year == 2024)                  [2]
df3 = df2.withColumn('tax', df2.amount*0.18)      [3]
df4 = df3.groupBy('city').sum('amount', 'tax')    [4] <- SHUFFLE
df4.write.parquet('s3://output/')                 [5] <- ACTION

GENERATED DAG:
[1] Read Parquet from S3
      |  narrow dependency (no shuffle needed here)
      v
[2] Filter: year == 2024              STAGE 1
      |  narrow dependency            All tasks run on same node
      v
[3] WithColumn: add tax column
      |
------+------ STAGE BOUNDARY (wide dependency / shuffle!) ------
      v
[4] GroupBy city -> Sum amount, tax   STAGE 2
      |  wide dependency              Data moves between nodes!
      v
[5] Write output to S3
```

6.2 Narrow vs Wide Transformations

The most critical concept for understanding when shuffles happen — and how to minimize them.

Narrow Transformation (No Shuffle)	Wide Transformation (Shuffle!)
Each output partition needs ONE input partition	Output partition needs data from MULTIPLE inputs
No data movement across the network	Data moves across the network -- expensive!
Stays within the same Stage in the DAG	Creates a new Stage boundary in the DAG
filter(), select(), map(), withColumn()	groupBy(), join(), distinct(), repartition()
flatMap(), union(), limit()	orderBy(), cogroup(), intersection()
Example: df.filter(city == 'Mumbai')	Example: df.groupBy('city').count()

6.3 The Shuffle — PySpark's Most Expensive Operation

Sateesh Says 💡

Shuffle is like this: 200 Zomato delivery boys across Hyderabad suddenly need to SWAP all their orders.

Every boy drives to a central hub, drops orders, picks up new ones, drives back. Absolute traffic chaos!

That is exactly what happens during a PySpark shuffle — massive data movement across the network.

Every groupBy, join, and distinct causes a shuffle. MINIMIZE shuffles = FASTER PySpark code. Period.

Shuffle — What Actually Happens on the Network

```
BEFORE SHUFFLE: 4 Partitions across 4 Nodes, groupBy('city')
Node 1, Part 0: [Mumbai, Delhi, Hyd]
Node 2, Part 1: [Mumbai, Chennai, Delhi]
Node 3, Part 2: [Hyd, Mumbai, Chennai]
Node 4, Part 3: [Delhi, Hyd, Mumbai]

SHUFFLE (all rows of same city must go to one reducer):
All Mumbai rows  ->  Reducer on Node 1
All Delhi rows   ->  Reducer on Node 2
All Hyderabad    ->  Reducer on Node 3
All Chennai      ->  Reducer on Node 4

Network traffic: Nodes 2, 3, 4 send Mumbai rows to Node 1.
Nodes 1, 3, 4 send Delhi rows to Node 2. And so on.
ALL 4 nodes are sending data to ALL other nodes. Network chaos!

AFTER SHUFFLE: Each node has all rows for its assigned city.
Node 1: sum all Mumbai rows. Node 2: sum all Delhi rows. Done!
```

```
Cost: Network I/O + Disk spill if data too big + Serialization.
Best practice: Broadcast small tables to AVOID join shuffles.
```

Module 7: SparkSession & Configuration

Every PySpark program starts with SparkSession. Configuring it correctly is the difference between a job that runs in 10 minutes versus 2 hours.

7.1 SparkSession — Your Entry Point

SparkSession — Minimal and Production Setup

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Minimal setup (local mode -- for learning on laptop)
spark = SparkSession.builder \
    .appName('LevelUp_PySpark_Demo') \
    .master('local[*]') \      # Use all CPU cores locally
    .getOrCreate()              # Get existing or create new

# Production cluster setup (Databricks / YARN)
spark = SparkSession.builder \
    .appName('Flipkart_Orders_ETL') \
    .config('spark.executor.memory', '8g') \
    .config('spark.executor.cores', '4') \
    .config('spark.sql.shuffle.partitions', '400') \
    .config('spark.driver.memory', '4g') \
    .config('spark.executor.instances', '20') \
    .enableHiveSupport() \     # If using Hive metastore
    .getOrCreate()

sc = spark.sparkContext         # Access underlying SparkContext
spark.stop()                    # Always stop when done!
```

7.2 Key Configuration Parameters to Tune

Config Parameter	Default	Recommended Setting
spark.sql.shuffle.partitions	200	2-4x number of CPU cores in cluster
spark.executor.memory	1g	4g to 16g depending on workload
spark.executor.cores	1	4-8 cores per executor (sweet spot)
spark.driver.memory	1g	4g to 8g (more for large collect())
spark.executor.instances	2	(total cores) / executor.cores
spark.default.parallelism	8	Same as shuffle.partitions value
spark.memory.fraction	0.6	0.6 to 0.8 for memory-heavy jobs

Module 8: Interview Cheat Sheet

Sateesh Says 💡

These are the exact questions that appear in TCS Digital, Infosys Specialist, Amazon DE, and Wipro PySpark interviews.

I have trained 5,000+ students through these. Nail these 12 questions and you are ready for any PySpark interview!

8.1 Top 12 PySpark Interview Questions

Q1 What is PySpark? How is it different from Pandas?

PySpark is Python's API for Apache Spark — processes data across a cluster. Pandas runs on a single machine in RAM. PySpark handles TB/PB scale; Pandas handles up to ~10 GB. PySpark is lazy; Pandas executes immediately.

Q2 Explain PySpark Architecture.

Driver Program coordinates everything (creates DAG, sends Tasks). Cluster Manager allocates resources. Executors on Worker Nodes run actual Tasks. SparkSession is the entry point. Data is split into Partitions processed in parallel.

Q3 What is the difference between Transformation and Action?

Transformations (filter, groupBy, join) are LAZY — they build the execution plan but don't run. Actions (show, count, write) TRIGGER execution. Spark waits for an Action, then optimizes and runs the full plan at once.

Q4 What is a DAG in Spark?

Directed Acyclic Graph — Spark's execution plan. Shows all transformations as nodes with directed edges. No loops. Created by Driver when an Action triggers. Optimized by Catalyst. Divided into Stages at shuffle boundaries.

Q5 What is a shuffle and why is it expensive?

Shuffle = movement of data between partitions across the network. Triggered by groupBy, join, distinct. Expensive because of: network I/O, disk spill, serialization overhead. Minimize shuffles for maximum PySpark performance.

Q6 Difference between repartition() and coalesce()?

repartition(): full shuffle, can increase or decrease partitions, even distribution. coalesce(): no shuffle, can only decrease partitions, may be uneven. Use coalesce() to reduce output files; use repartition() to fix data skew before heavy operations.

Q7 What is lazy evaluation in Spark?

Spark does not execute transformations immediately — builds a logical plan. Only when an Action triggers does it execute. Allows Catalyst Optimizer to see the full picture and optimize (column pruning, predicate pushdown, join reordering).

Q8 What is data skew and how do you handle it?

Data skew = some partitions have far more data than others. One Task runs for hours while 99 others finish in minutes. Solutions: repartition on a more even column, salting technique, using broadcast join for small dimension tables.

Q9 What is the difference between cache() and persist()?

cache() stores DataFrame in memory (with disk fallback). persist() lets you specify the storage level: MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, and replicated variants. Use both when accessing the same DataFrame multiple times in your code.

Q10 What is a broadcast join?

When one DataFrame is small (< 10 MB), Spark broadcasts a copy to every Executor. The join then happens locally with no shuffle. Usage: df_large.join(broadcast(df_small), 'city'). Removes the most expensive operation — shuffle — entirely!

Q11 Explain narrow vs wide transformations.

Narrow: output partition depends on one input partition only. No shuffle. Examples: filter, map, select. Wide: output depends on multiple input partitions. Causes shuffle. Examples: groupBy, join, distinct. Wide transformations create Stage boundaries.

Q12 What is the Catalyst Optimizer?

Spark's built-in query optimizer for DataFrames and SQL. Takes your logical plan, applies optimization rules (predicate pushdown, column pruning, join reordering), generates physical execution plan. Key reason DataFrame code outperforms equivalent RDD code.

Module 9: Quick Reference Card

PySpark Hierarchy & Critical Numbers

```
SparkSession > Job > Stage > Task > Partition
(1 per app)   (per action) (per shuffle) (per core) (per chunk)

TRANSFORMATIONS (LAZY -- build the plan, no execution):
Narrow:  filter, select, map, flatMap, withColumn, union, limit
Wide:    groupBy, join, distinct, repartition, orderBy, cogroup

ACTIONS (trigger execution -- Spark starts working!):
show(), count(), collect(), take(n), first(), write()
saveAsTable(), saveAsParquet(), foreach(), toPandas()

KEY DEFAULTS TO KNOW:
spark.sql.shuffle.partitions   = 200  (tune to 2-4x your cores)
spark.default.parallelism      = total executor cores
Broadcast auto-threshold       = 10 MB
HDFS read partition size       = based on block size (128 MB)

STORAGE LEVELS (for persist()):
MEMORY_ONLY         Fastest. Evicts partitions if RAM full.
MEMORY_AND_DISK     Falls back to disk if RAM full. (default)
DISK_ONLY           For very large DataFrames. Saves RAM.
MEMORY_ONLY_2       Replicates to 2 nodes. High availability.

WHEN TO USE WHAT:
Small table join  -> broadcast() -- eliminates shuffle entirely
Many output files -> coalesce() -- reduce before writing
Skewed data       -> repartition() -- redistribute evenly
Repeated use      -> cache() / persist() -- avoid recomputation
```

Sateesh Says 💡

You have just completed the foundation of PySpark. Here is your roadmap from here:
Step 1: Install PySpark locally (pip install pyspark) and run your first SparkSession today.
Step 2: Practice the 5 most common transformations: filter, groupBy, join, withColumn, select.
Step 3: Open Spark UI (localhost:4040) while a job runs and watch the DAG come to life!
Step 4: Always think about partitions, shuffles, and lazy evaluation when writing code.
Remember: Understanding WHY makes you unstoppable. The API is just syntax.
All the best for your PySpark journey! -- Sateesh Pabbathi, MCT | www.levelupedu.net